CGA PRACTICALS CODE

Practical's

1A:-(Line)

```
import turtle
t = turtle.Turtle()
t.forward(200)
turtle.done()
```

1B:-(Circle)

```
import turtle
t = turtle.Turtle()
t.circle(100)
turtle.done()
```

1C:-(Square)

```
import turtle
t = turtle.Turtle()
for i in range(4):
    t.forward(100)   # side length
    t.right(90)
turtle.done()
```

1D:-(Rectangle)

```
import turtle
t = turtle.Turtle()
# Length = 150, Breadth = 80
for i in range(2):
```

```
    t.forward(150)
    t.right(90)
    t.forward(80)
    t.right(90)
turtle.done()
```

1E:-(Polygon)

```
import turtle
t = turtle.Turtle()
sides = 8
length = 45
for i in range(sides):
    t.forward(length)
    t.right(360 / sides)
turtle.done()
```

1F**:-(Hexagon)**

```
import turtle
t = turtle.Turtle()
for i in range(6):
    t.forward(100)
    t.right(60)
turtle.done()
```

Practical 2:- DDA Line drawing algorithm CGA practical

```
import turtle
```

```
s = turtle.Screen()
s.setup(400, 400)
s.tracer(0)
t = turtle.Turtle()
t.hideturtle()
t.penup()
def dda(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    steps = int(max(abs(dx), abs(dy)))
    x = x1
    y = y1
    for _ in range(steps + 1):
        t.goto(round(x), round(y))
        t.dot(3)
        x += dx / steps
        y += dy / steps
# single slanted line inside canvas
dda(-120, 80, 120, -60)
s.update()
turtle.done()
```

Practical 3:- Bresenham line drawing algorithms CGA Practical

```
import turtle
def bresenham(x1, y1, x2, y2):
```

```
    turtle.penup()
    turtle.goto(x1, y1)
    turtle.pendown()
    dx = x2 - x1
    dy = y2 - y1
    d = 2*dy - dx
    incE = 2*dy       # move straight (E)
    incNE = 2*(dy - dx)  # move diagonal (NE)
    x = x1
    y = y1
    while x <= x2:
        turtle.goto(x, y)
        if d <= 0:
            d = d + incE     # move right
        else:
            d = d + incNE    # move diagonal (up)
            y = y + 1
        x = x + 1
# Example call
bresenham(0, 0, 200, 80)
turtle.done()
```

Practical 4 – Midpoint Circle Algorithm

```
Import turtle
Def midpoint_circle(xc, yc, r):
    T = turtle.Turtle()
```

```
t.speed(0)
t.hideturtle()
x = r
y = 0
p = 1 – r
def draw_point(x, y):
    t.penup()
    t.goto(xc + x, yc + y)
    t.dot(3)
def plot_circle_points(x, y):
    draw_point(x, y)
    draw_point(-x, y)
    draw_point(x, -y)
    draw_point(-x, -y)
    draw_point(y, x)
    draw_point(-y, x)
    draw_point(y, -x)
    draw_point(-y, -x)
# Initial points
Plot_circle_points(x, y)
While x > y:
    Y += 1
    If p <= 0:
        P = p + 2 * y + 1
    Else:
        X -= 1
        P = p + 2 * y – 2 * x + 1
```

```
        If x < y:
            Break
        Plot_circle_points(x, y)
# Draw
Screen = turtle.Screen()
Screen.setup(600, 600)
Midpoint_circle(0, 0, 120)
Turtle.done()
```

Practical 5:- Scaling

```
Import graphics
Win = graphics.GraphWin("Scale Square", 600, 600)
Print("Corner 1")
X1 = float(input("Enter x: "))
Y1 = float(input("Enter y: "))
```

```
C1 = graphics.Point(x1, y1)
Print("Corner 2")
X2 = float(input("Enter x: "))
Y2 = float(input("Enter y: "))
C2 = graphics.Point(x2, y2)
S = graphics.Rectangle(c1, c2)
s.draw(win)
sx = float(input("Scaling Factor sx: "))
sy = float(input("Scaling Factor sy: "))
# Scale the coordinate
```

Practical 6 :-Flood fill

```
Import pygame
Import sys
# Example 2D array
Field = [
    [1, 0, 2],
    [0, 1, 0],
```

```
    [2, 0, 1]
]
CELL_SIZE = 60
WIDTH  = len(field[0]) * CELL_SIZE
HEIGHT = len(field)    * CELL_SIZE
Def draw_field(screen, field):
    Font = pygame.font.SysFont(None, 40)
    For y in range(len(field)):
        For x in range(len(field[0])):
            Value = field[y][x]
            # cell rect
            Rect = pygame.Rect(x * CELL_SIZE, y * CELL_SIZE, CELL_SIZE, CELL_SIZE)
            Pygame.draw.rect(screen, (200, 200, 200), rect, 1)
            # draw value
            Text = font.render(str(value), True, (255, 255, 255))
            Screen.blit(text, (x * CELL_SIZE + 20, y * CELL_SIZE + 10))
Pygame.init()
Screen = pygame.display.set_mode((WIDTH, HEIGHT))
Pygame.display.set_caption("Grid Display")
Screen.fill((0, 0, 0))
Draw_field(screen, field)
Pygame.display.flip()
While True:
    For event in pygame.event.get():
        If event.type == pygame.QUIT:
            Pygame.quit()
            Sys.exit()
```

Practical 7:- Python program to draw smile

```
# face emoji using turtle
Import turtle
# turtle object
Pen = turtle.Turtle()
# function for creation of eye
Def eye(col, rad):
        Pen.down()
        Pen.fillcolor(col)
```

```
        Pen.begin_fill()
        Pen.circle(rad)
        Pen.end_fill()
        Pen.up()

# draw face
Pen.fillcolor('yellow')
Pen.begin_fill()
Pen.circle(100)
Pen.end_fill()
Pen.up()

# draw eyes
Pen.goto(-40, 120)
Eye('white', 15)
Pen.goto(-37, 125)
Eye('black', 5)
Pen.goto(40, 120)
Eye('white', 15)
Pen.goto(40, 125)
Eye('black', 5)

# draw nose
Pen.goto(0, 75)
Eye('black', 8)

# draw mouth
```

```
Pen.goto(-40, 85)
Pen.down()
Pen.right(90)
Pen.circle(40, 180)
Pen.up()

# draw tongue
Pen.goto(-10, 45)
Pen.down()
Pen.right(180)
Pen.fillcolor('red')
Pen.begin_fill()
Pen.circle(10, 180)
Pen.end_fill()
Pen.hideturtle()
```

Practical 8:- Drawing nested squares using loops.

```
Import turtle
Vivax=turtle.Turtle()
Def Multiple_Squares(length, colour):
  Vivax.pencolor(colour)
  Vivax.pensize(4)
  Vivax.forward(length)
  Vivax.right(90)
  Vivax.forward(length)
  Vivax.right(90)
  Vivax.forward(length)
```

```
  Vivax.right(90)
  Vivax.forward(length)
  Vivax.right(90)
  Vivax.setheading(360)
  Vivax.ht()
For I in range(50,110,10):
  Multiple_Squares(I,”red”)
```

Practical 9:- Simple Bouncing Ball

```
Import pygame
# Initialize and set up window
Pygame.init()
Screen = pygame.display.set_mode((600, 400))
Clock = pygame.time.Clock()

# Ball variables: [x, y] position and [x_speed, y_speed]
Pos = [300, 200]
Speed = [5, 5]
Radius = 15
While True:
```

```
# 1. Check for exit
For event in pygame.event.get():
    If event.type == pygame.QUIT:
        Pygame.quit()
        Exit()

# 2. Move the ball
Pos[0] += speed[0]
Pos[1] += speed[1]

# 3. Bounce logic (Collision detection)
If pos[0] <= radius or pos[0] >= 600 – radius:
    Speed[0] *= -1  # Reverse horizontal direction
If pos[1] <= radius or pos[1] >= 400 – radius:
    Speed[1] *= -1  # Reverse vertical direction

# 4. Draw everything
Screen.fill((30, 30, 30)) # Dark background
Pygame.draw.circle(screen, (255, 100, 100), pos, radius
Pygame.display.flip()
Clock.tick(60) # Run at 60 frames per second
```

Practical 10 :- Final Project

```
Import pygame

# --- Initialization ---

Pygame.init()

Screen = pygame.display.set_mode((800, 400))

Clock = pygame.time.Clock()


# Colors

SKY_BLUE = (135, 206, 235)

GRASS_GREEN = (34, 139, 34)

ROAD_GREY = (50, 50, 50)

SUN_GOLD = (255, 215, 0)


# Variables
```

```
Sun_y = 450          # Start below the horizon
Tree_x = 800         # Start at the right edge
Car_x = 100          # Fixed car position

While True:
    For event in pygame.event.get():
        If event.type == pygame.QUIT:
            Pygame.quit()
            Exit()

    # --- Logic ---
    # 1. Move Sun up (Rising)
    If sun_y > 100:
        Sun_y -= 0.5

    # 2. Move Tree (Looping background)
    Tree_x -= 7
    If tree_x < -50:
        Tree_x = 800

    # --- Drawing ---
    # Draw Sky & Grass
    Screen.fill(SKY_BLUE)
    Pygame.draw.rect(screen, GRASS_GREEN, (0, 250, 800, 150))

    # Draw Sun
    Pygame.draw.circle(screen, SUN_GOLD, (700, int(sun_y)), 40)
```

```
# Draw Road
Pygame.draw.rect(screen, ROAD_GREY, (0, 300, 800, 60))
Pygame.draw.line(screen, (255, 255, 255), (0, 330), (800, 330), 2) # Lane line

# Draw Tree (Trunk and Leaves)
Pygame.draw.rect(screen, (101, 67, 33), (tree_x, 220, 20, 80))
Pygame.draw.circle(screen, (0, 100, 0), (tree_x + 10, 220), 35)

# Draw Car (Body and Wheels)
Pygame.draw.rect(screen, (200, 0, 0), (car_x, 280, 100, 40))   # Car body
Pygame.draw.rect(screen, (200, 0, 0), (car_x + 20, 260, 60, 25)) # Car roof
Pygame.draw.circle(screen, (0, 0, 0), (car_x + 20, 320), 12)  # Wheel 1
Pygame.draw.circle(screen, (0, 0, 0), (car_x + 80, 320), 12)  # Wheel 2

Pygame.display.flip()
Clock.tick(60)
```

Practical 11 :- Interactive Click-to-Spawn

```
Import pygame
Pygame.init()
Screen = pygame.display.set_mode((800, 600))
Clock = pygame.time.Clock()

# List to store the positions of our shapes
Circles = []

While True:
    Screen.fill((30, 30, 30)) # Clear screen each frame

    # 1. Check for Mouse Events
    For event in pygame.event.get():
        If event.type == pygame.QUIT:
            Pygame.quit()
            Exit()
```

```
    # If the user clicks the mouse button
    If event.type == pygame.MOUSEBUTTONDOWN:
        # Get the (x, y) position of the click
        Pos = pygame.mouse.get_pos()
        # Save it to our list
        Circles.append(pos)

# 2. Draw all saved circles
For c_pos in circles:
    Pygame.draw.circle(screen, (0, 255, 200), c_pos, 25)

# 3. Draw a “ghost” circle at the current mouse position
Mouse_pos = pygame.mouse.get_pos()
Pygame.draw.circle(screen, (255, 255, 255), mouse_pos, 25, 1) # Width=1 for outline
Pygame.display.flip()
Clock.tick(60)
```

Practical 12 :- Simple Collision

```
Import pygame
Pygame.init()
Screen = pygame.display.set_mode((600, 400))
Clock = pygame.time.Clock()

# Create two Rect objects: pygame.Rect(x, y, width, height)
Player_rect = pygame.Rect(0, 0, 50, 50)
Goal_rect = pygame.Rect(275, 175, 50, 50)

While True:
    For event in pygame.event.get():
        If event.type == pygame.QUIT:
            Pygame.quit()
            Exit()

    # 1. Update: Move player rect to mouse position
    Player_rect.center = pygame.mouse.get_pos()
```

```
# 2. Collision Detection
# Check if player_rect touches goal_rect
If player_rect.colliderect(goal_rect):
    Color = (0, 255, 0) # Green if touching
Else:
    Color = (255, 0, 0) # Red if not touching

# 3. Draw
Screen.fill((30, 30, 30))
Pygame.draw.rect(screen, color, goal_rect)        # Draw goal
Pygame.draw.rect(screen, (255, 255, 255), player_rect) # Draw player
Pygame.display.flip()
Clock.tick(60)
```

Practical 13 :- Multiple Object Animation

```
Import pygame
Import random
# Setup
Pygame.init()
WIDTH, HEIGHT = 800, 600
Screen = pygame.display.set_mode((WIDTH, HEIGHT))
Clock = pygame.time.Clock()

Class Ball:
    Def __init__(self):
        # Each ball gets its own random properties
        Self.radius = random.randint(10, 30)
        Self.x = random.randint(self.radius, WIDTH – self.radius)
        Self.y = random.randint(self.radius, HEIGHT – self.radius)
        Self.speed_x = random.choice([-4, -3, 3, 4])
        Self.speed_y = random.choice([-4, -3, 3, 4])
        Self.color = (random.randint(50, 255), random.randint(50, 255), random.randint(50, 255))

    Def move(self):
        # Update position
```

```
        Self.x += self.speed_x
        Self.y += self.speed_y

        # Bounce logic
        If self.x <= self.radius or self.x >= WIDTH – self.radius:
            Self.speed_x *= -1
        If self.y <= self.radius or self.y >= HEIGHT – self.radius:
            Self.speed_y *= -1

    Def draw(self, surface):
        Pygame.draw.circle(surface, self.color, (int(self.x), int(self.y)), self.radius)

# Create a list containing 10 Ball objects
Balls = [Ball() for _ in range(10)]

While True:
    For event in pygame.event.get():
        If event.type == pygame.QUIT:
            Pygame.quit()
            Exit()
    Screen.fill((20, 20, 20))

    # Update and draw every ball in the list
    For ball in balls:
        Ball.move()
        Ball.draw(screen)
    Pygame.display.flip()
```

```
Clock.tick(60)
```